Document Databases Explained

embed vs reference, and MongoDB's limits

Watch the 4-min video

In one line: a document database stores each entity as one JSON-like document with its own shape. Embed data that is read together, owned and bounded; reference data that is shared, large, unbounded or changes on its own; and keep documents well under the size limit.

1 The problem: products that don't share a shape

Your catalog sells shoes, laptops and sofas. Shoes have sizes and colors. Laptops have a CPU, RAM and a screen size. Sofas have dimensions and fabric. In one relational table, that is **a hundred mostly empty columns**, or an awkward attributes table (one row per product per attribute). A document database stores each product as **its own JSON document**.

One relational table

name	size	color	cpu	ram	screen	fabric	...
Run X	✓	✓	null	null	null	null	
Book 14	null	null	✓	✓	✓	null	
Sofa 3	null	✓	null	null	null	✓	

~100 columns, mostly null
or an awkward attributes table

One document per product

```
{ name: "Run X",
  sizes: [8, 9],
  colors: ["red"] }
```

```
{ name: "Book 14",
  cpu: "M3",
  ram_gb: 16 }
```

```
{ name: "Sofa 3",
  dims: {w: 210},
  fabric: "linen" }
```

each with its own shape

Figure 1. The relational version wastes columns and fights the data. In a document store each product carries only the fields it has.

2 What a document database is

It stores **JSON-like documents**, grouped into **collections** (roughly, tables). Each document can have its own shape, with **nested objects and arrays**. MongoDB stores **BSON**, a binary form of JSON that adds types such as dates and 64-bit integers; Azure Cosmos DB works similarly. Documents map naturally to the objects in your code, so there is less translation between the database and the app.

3 The key design question: embed or reference?

Embed related data inside the document when it is **read together**, **owned by the parent**, and **bounded in size**: a product's variants, an order's line items. One read returns everything.

Reference it by ID when the related data is **shared, large, unbounded, or changes independently**: a product's reviews, which can grow to thousands, or the customer on an order, who is shared by many orders and updates their address on their own schedule.

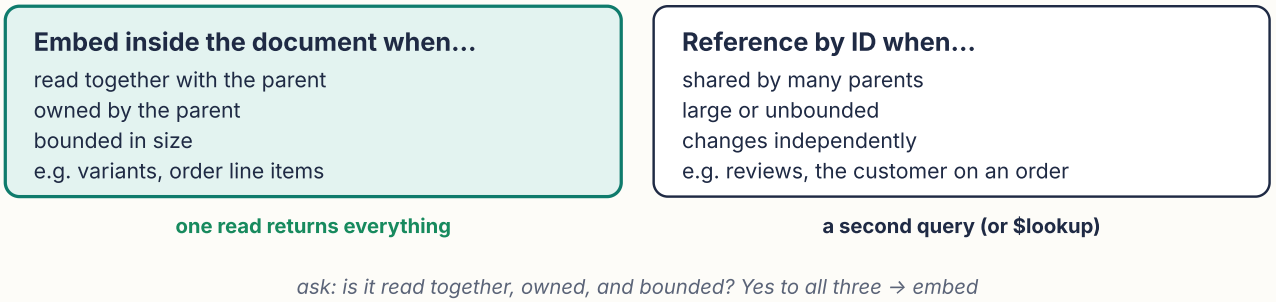


Figure 2. Three yes answers mean embed. Any one no (shared, unbounded, independently updated) means reference.

4 Example: the product catalog

Each product document has the common fields: ID, name, category and price. Then an `attributes` object that differs per category: sizes and colors for shoes, CPU and RAM for laptops. The **variants are embedded** as an array. The **reviews live in their own collection**, each referencing the product ID.

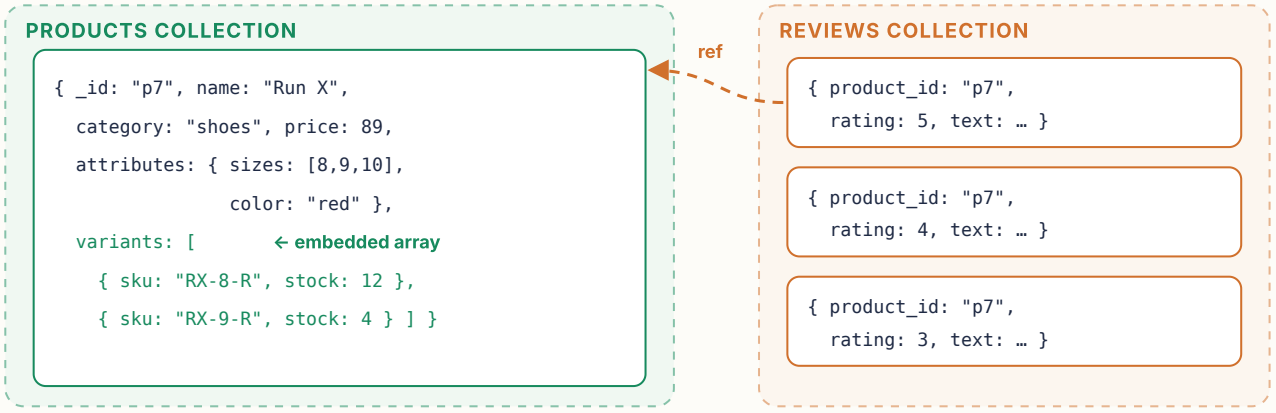


Figure 3. A product page is one read of the product (with variants) plus one indexed query for its latest reviews.

5 Indexes and aggregation

Document databases support **secondary indexes on any field**, including nested fields and arrays (a multikey index indexes every array element). Index `attributes.color` and you can filter shoes by color efficiently. **Without an index, a query scans the whole collection.**

Aggregation pipelines filter, group and compute in stages, for example the average rating per category:

```
db.products.createIndex({ category: 1, "attributes.color": 1 })
db.products.find({ category: "shoes", "attributes.color": "red" })

db.reviews.aggregate([
  { $match: { created: { $gte: ISODate("2026-01-01") } } },
  { $group: { _id: "$category", avgRating: { $avg: "$rating" } } },
  { $sort:  { avgRating: -1 } }
])   // assumes each review copies its product's category
```

6 Watch the limits

- **Size caps.** A MongoDB document can be at most **16 MB**; a Cosmos DB item **2 MB**. Unbounded arrays, like embedding every review, will eventually break.

- **Big documents are slow to change.** Every update rewrites more data, and every read ships more bytes.
- **Multi-document transactions** exist, but they are slower, and slower still across shards. Design most operations to touch **a single document**, which is always atomic.

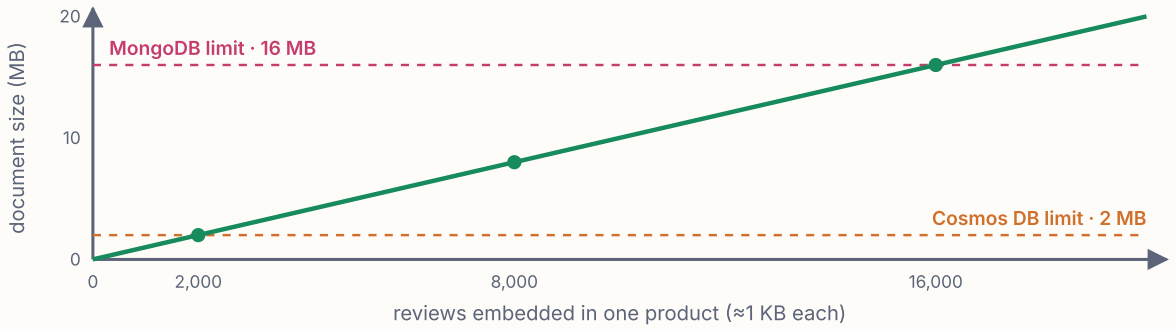


Figure 4. At about 1 KB per review, a Cosmos DB item hits its limit near 2,000 embedded reviews and a MongoDB document near 16,000. Reviews belong in their own collection.

WORKED EXAMPLE: EMBED THE LINE ITEMS?

- **Order line items:** read with the order, owned by it, and bounded (say at most 100 lines × 300 bytes = 30 KB). **Embed.**
- **Customer:** shared by hundreds of orders and edited separately. **Reference** by `customer_id`, and copy only what must not change (the shipping address at order time).
- **Product reviews:** a popular product gets 50,000 reviews × 1 KB = 50 MB, three times MongoDB's cap. **Reference.** If the page needs a preview, embed only the 5 newest and a review count.

7 When documents fit

They fit when entities are **self-contained and vary in shape**: catalogs, content, user profiles and event logs. They are weaker when data is **highly relational**, with many-to-many links queried in every direction. And **Postgres with JSONB** is a strong middle ground: flexible JSON columns, indexable with GIN, inside relational tables with joins and transactions.

COMMON MISTAKES

- Embedding an unbounded, growing array (reviews, comments, events) until the document hits the size cap.
- Querying fields with no index, so every request scans the whole collection.
- Modelling documents like normalized tables, then needing a join for every page.
- Relying on multi-document transactions for every operation instead of shaping documents so one write is enough.

8 Key terms

Document: a JSON-like record with its own shape

Collection: a group of documents (like a table)

BSON: MongoDB's binary JSON with extra types

Embed: store related data inside the document

Reference: store the related document's ID

Multikey index: an index on every element of an array

Aggregation pipeline: staged filter, group and compute

16 MB / 2 MB: MongoDB / Cosmos DB size limits

JSONB: Postgres binary JSON column type

9 Interview questions

Q1. When do you embed, and when do you reference? BASIC

Embed data that is read together, owned by the parent and bounded in size, like variants or line items. Reference data that is shared, large, unbounded or updated independently, like reviews or customers.

Q2. What happens if you query a field with no index? BASIC

A full collection scan: every document is read. On 10 million products that turns a 2 ms query into seconds. Index the fields you filter and sort on, including nested ones like attributes.color.

Q3. Why are single-document writes preferred? BASIC

A write to one document is always atomic, with no transaction coordination. Multi-document transactions work but are slower, and slower still across shards.

Q4. Design a blog with posts and comments in MongoDB. A post can get 20,000 comments. SCENARIO

Posts collection with title, body, author name and a comment count. Comments in their own collection with post_id and created, indexed on (post_id, created). Optionally embed the latest 3 comments in the post for the page preview.

Q5. Users can edit their display name. It is embedded in 2 million comments. Problem? SCENARIO

Yes: renaming means updating 2 million documents. Reference the user by ID and look the name up (or cache it), or accept the old name on old comments and update in the background. Embed only data that rarely changes.

Q6. Postgres JSONB or MongoDB for a product catalog with orders and payments? SCENARIO

Postgres with JSONB is a strong fit: a JSONB attributes column for the varying fields, normal tables for orders and payments, and real transactions and joins between them. Pick MongoDB if the catalog dominates, needs to shard, and rarely joins.

Q7. Your MongoDB updates get slower as documents grow, even with indexes. Why? TRICKY

Each update rewrites the whole document, and reads ship it all over the network. A 5 MB document costs 5 MB per change. Split out the growing part (history, reviews) into its own collection.

Q8. Is a document database “schemaless”? TRICKY

No. The schema moves into the application: your code still expects certain fields. Use schema validation (MongoDB $jsonSchema) and versioned document shapes so old and new documents can be read safely.

Next: SD-E-016 · Wide-Column Stores backendroadmap.com